MÓDULO 3 · ACTIVIDAD M3 · RETO 3

Un sistema que se degrada solo y se recupera con calma

Sistemas UltraSeguros S.A. falla en los picos de tráfico. Diseñamos en **AWS serverless** tres niveles de servicio (Full, Degradado y Mínimo) que cambian según la salud observada **minuto a minuto**, sin intervención humana.

Group 2 · Daniel Mazo · Freddy Aparicio · Camilo Céspedes
Diplomado en Arquitectura de Software y Cloud Computing · PUJ Cali · noviembre 2026

PUNTO DE PARTIDA

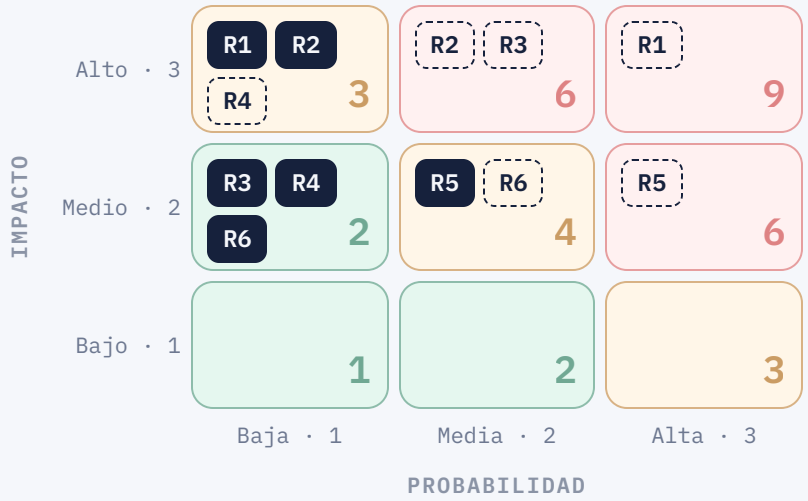
Priorizamos la disponibilidad: responder siempre, aunque sea con menos

La primera versión subía y bajaba de nivel de inmediato y, con el script oficial, oscilaba. Al releer el enunciado entendimos que el reto es un **sistema de recuperación**: pide volver a Nivel 2 cuando los indicadores estén «saludables» y a Nivel 1 solo cuando estén «completamente estables». No basta con reaccionar: hay que medir la estabilidad para recorrer los niveles en orden. Por eso, después de degradar, el sistema exige una **racha de peticiones sin error** para recuperarse: **30 para volver a N2 y 40 para N1**, siempre de a un nivel.



Script oficial del docente: 140 peticiones, una cada 3 s, con 5 · 0 · 15 · 0 · 15 · 0 · 0 errores por «minuto» (bloque de 20 peticiones)

Seis riesgos de la resiliencia, y cómo los bajamos



R1 antes de mitigar · R1 ahora · 1–2 bajo · 3–4 medio · 6–9 alto

Matriz de riesgos (técnica del módulo): riesgo = impacto × probabilidad, de 1 a 3 cada uno. Mitigar no siempre baja el impacto: casi siempre baja la probabilidad.

Resultado: ningún riesgo queda alto. Los seis suman 34 antes de mitigar y 16 después; los tres que quedan en medio son de alto impacto (R1, R2) o dependen del script (R5).

R1 — El sistema oscila entre niveles

9 → 3

Ejemplo: pasó en la primera versión. El minuto 1 llegaba limpio y devolvía el sistema a N1; el minuto 2 traía 15 errores y lo mandaba directo a N3: 1-2-**1-3**-2-**3**-2-1.

Solución: para subir exige rachas sin error (30 peticiones para N2 y 40 para N1), de a un nivel. Un bloque limpio ya no lo devuelve a N1.

R2 — Dos instancias responden niveles distintos

6 → 3

Ejemplo: dos peticiones caen en dos instancias de Lambda, cada una con su memoria: una podría responder «Nivel 1» y la otra «Nivel 3».

Solución: nivel, número de petición y racha viven en un solo ítem de DynamoDB, con escritura condicional por versión.

R3 — El servicio de un nivel falla o se cuelga

6 → 2

Ejemplo: la Lambda del Nivel 1 no responde. Sin límite, el cliente queda esperando y los reintentos automáticos alargan la espera.

Solución: se corta a los 3 s, sin reintentos: responde el mensaje del Nivel 3; el error cuenta en el bloque y rompe la racha.

R4 — DynamoDB no responde

3 → 2

Ejemplo: el router no puede leer el estado, así que no sabe en qué nivel está.

Solución: asume lo peor y responde «Nivel 3: Sistema bajo mantenimiento» (503), nunca un error crudo.

R5 — Las rachas de 30 y 40 dependen del script

6 → 4

Ejemplo: con otro patrón de carga, la racha llega antes o después y la secuencia cambia.

Solución: calibradas con 5 peticiones de margen y configurables (`RACHA_N2`, `RACHA_N1`) sin tocar el código.

R6 — El ítem único de estado es un punto caliente

4 → 2

Ejemplo: con mucho tráfico concurrente, todas las instancias escriben el mismo ítem y chocan.

Solución: escritura condicional por versión, hasta 5 reintentos: la que pierde relee y recalcula.

01 · DECISIONES QUE TOMAMOS

Seis decisiones, cada una con su porqué

1

El «minuto» anterior decide el nivel

Es la regla del docente. El «minuto» es el del k6, un bloque de 20 peticiones: la latencia ya no corre las ventanas, a cambio de depender del ritmo del cliente.

2

Degradar de golpe, recuperar con calma

Baja de nivel apenas hay errores; para subir exige 30 peticiones seguidas sin error (N3→N2) y 40 (N2→N1). Así no oscila.

3

Un router detrás del API Gateway

El gateway no sabe enrutar según un estado. El router decide y llama solo al servicio del nivel vigente.

4

Estado compartido en DynamoDB

Las instancias de Lambda no comparten memoria. El estado vive en un solo ítem con versión: todas responden el mismo nivel.

5

Ante la duda, Nivel 3

Si el estado no se puede leer o un servicio falla, respondemos al mínimo con un mensaje claro, nunca con un error crudo.

6

Infraestructura como código

Toda la arquitectura vive en una plantilla: nombres, permisos y parámetros en un solo lugar.

02 · RESTRICCIONES

Lo que no se negocia, y cómo lo cumplimos

1

Solo servicios de AWS

API Gateway, Lambda, DynamoDB y CloudWatch.

2

El script de k6 no se modifica

Solo cambia la URL, y lo comprobamos en cada corrida.

3

API Gateway como entrada única

Toda petición entra por `/service-api` y el router la lleva al nivel vigente.

4

Mensajes exactos por nivel

Cada servicio responde el texto que pide el enunciado, incluidos los del Nivel 3.

5

Sin plantilla inicial

Diseño y código desde cero.

6

Una cuenta de AWS para el equipo

Un responsable despliega y el panel entra con un usuario de solo lectura: nadie más puede modificar la infraestructura.

03 · ATRIBUTOS DE CALIDAD PRIORIZADOS

Tres atributos guiaron el diseño

PRIORIDAD 1

Disponibilidad

QUÉ SIGNIFICA AQUÍ

Aunque haya fallas, **cada petición recibe una respuesta con sentido**. En el peor caso, «Nivel 3: Operación al mínimo», nunca un error del servidor.

CÓMO LO CUMPLIMOS

Con errores, el sistema **reduce funciones en vez de caerse**. Si no conoce su estado, opera en Nivel 3.

EJEMPLO

Si en un minuto llegan 10 errores, el sistema no se cae: el minuto siguiente pasa a Nivel 3 y responde **«Operación al mínimo»**. El cliente siempre recibe una respuesta.

PRIORIDAD 2

Observabilidad

QUÉ SIGNIFICA AQUÍ

Cada cambio de nivel **queda registrado y explicado**.

CÓMO LO CUMPLIMOS

Un registro por cambio, con los errores que lo causaron, y métricas en CloudWatch.

EJEMPLO

Cuando pasa de Nivel 2 a Nivel 3 deja escrito **cuándo, de qué nivel a cuál y cuántos errores lo causaron**. Cualquiera puede explicar cada cambio sin adivinar.

PRIORIDAD 3

Modificabilidad

QUÉ SIGNIFICA AQUÍ

Cambiar la regla **sin tocar el código**.

CÓMO LO CUMPLIMOS

Umbrales (5 y 10 errores) y rachas (30 y 40 peticiones) son parámetros del despliegue.

EJEMPLO

Si el docente pide otros umbrales u otras rachas, **se cambia el valor en el despliegue** y el código queda igual.

A costa de tiempo de reacción. Exigir rachas de 30 y 40 peticiones sin error evita que el sistema oscile y asegura la secuencia 1-2-3-2-1, pero tarda más en volver a Nivel 1. Y cada degradación se aplica al «minuto» siguiente, no al instante.

Detectar, degradar y reintroducir

1 Detectar la falla

¿Algo está fallando?

Monitor
El router mira el resultado de cada petición

Condition monitoring
Cuenta los errores de cada «minuto» (20 peticiones) contra los umbrales 5 y 10

Exception detection
Un error del servicio o una espera de más de 3 s cuentan como falla

2 Degradar sin caerse

¿Cómo sigo respondiendo?

Graceful degradation
Apaga funciones: N2 deja de analizar y notificar; N3 solo responde

Reconfiguration
El router envía cada petición al servicio del nivel vigente

Exception handling
Toda falla termina en un mensaje claro, nunca en un error crudo

3 Reintroducir con calma

¿Cuándo vuelvo a la normalidad?

Histéresis
Para recuperarse exige una racha de peticiones sin error: un bloque bueno no basta

`punto de sensibilidad · 5/10 y 30/40`

Shadow (adaptada)
Devuelve las funciones de a un nivel: N2 es la prueba antes de N1. Si vuelven los errores, degrada otra vez

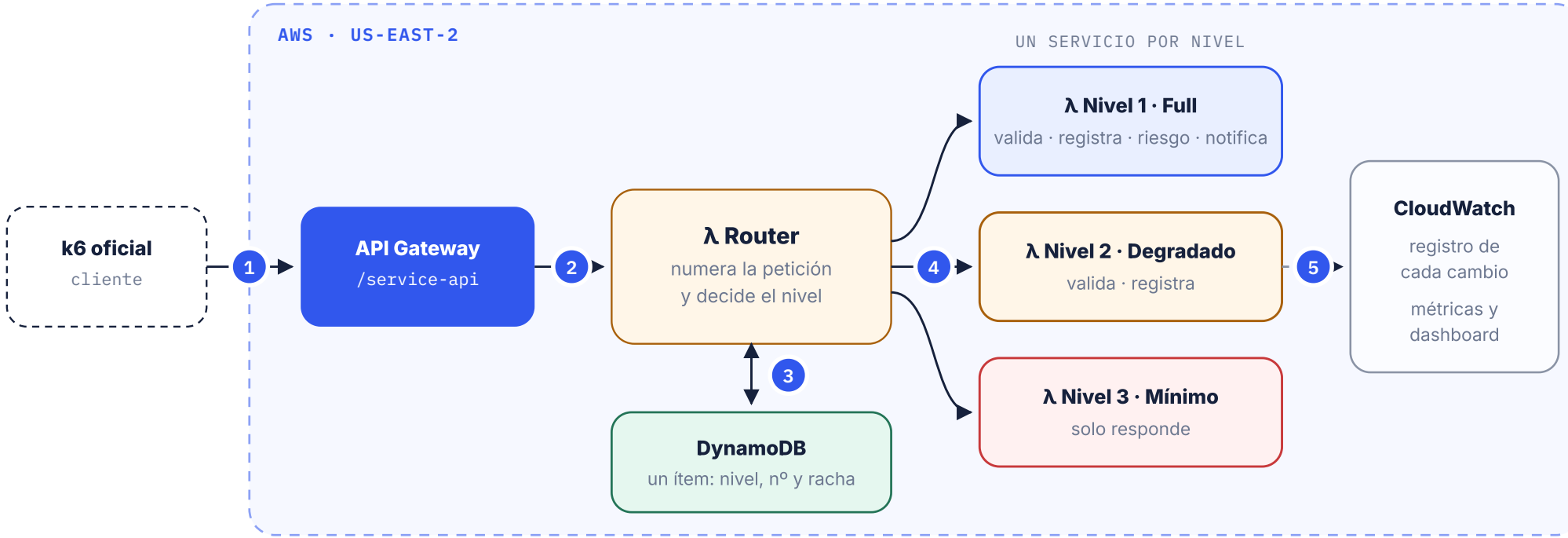
N3 — racha 30 — N2 — racha 40 — N1

Escenario de disponibilidad FUENTE k6 ESTÍMULO ≥5 o ≥10 errores en 20 peticiones ARTEFACTO router y Lambdas ENTORNO operación normal, 1 petición cada 3 s
RESPUESTA degrada a N2/N3 y reintroduce de a un nivel MEDIDA 140/140 respondidas · 1-2-3-2-1

Se parece a un circuit breaker, con un nivel intermedio N1 ≈ cerrado · N2 · degrada en vez de cortar · N3 ≈ abierto con ventanas de 20 peticiones

05 · ARQUITECTURA · VISTA DE CONTENEDORES

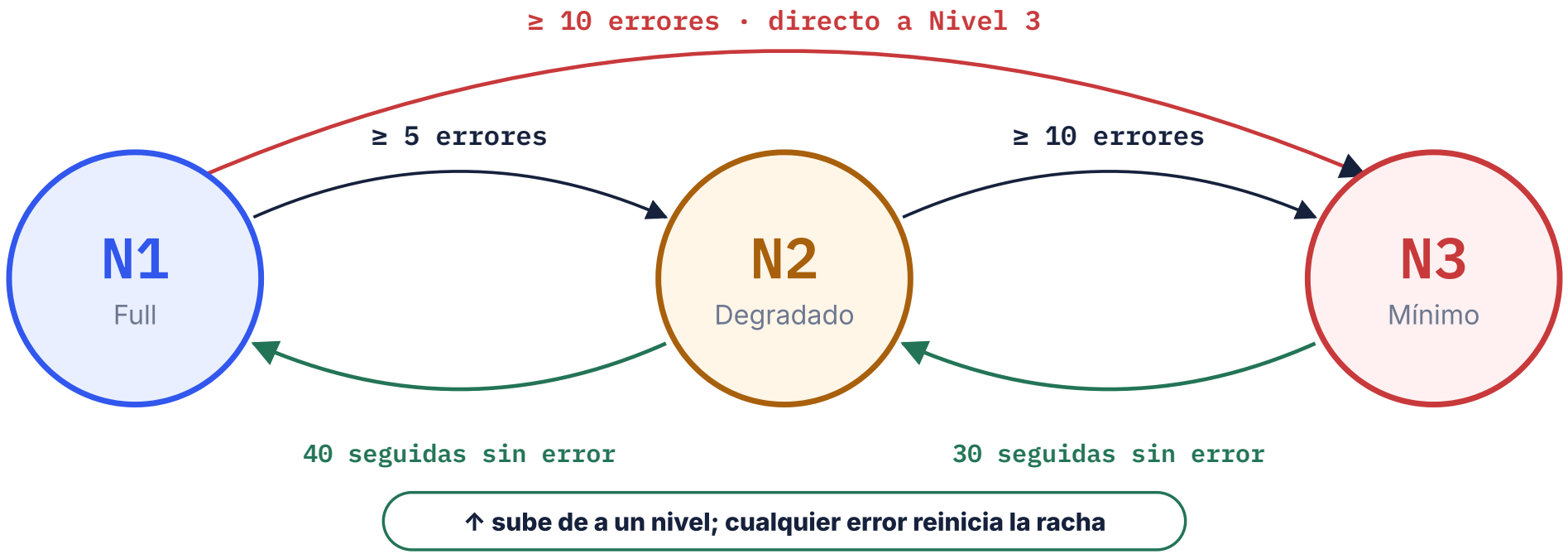
Un gateway, un router que decide y tres servicios



1. k6 envía la petición por HTTPS
2. API Gateway la pasa al router
3. El router numera la petición, evalúa el bloque anterior y la racha, y guarda el estado en DynamoDB
4. Invoca solo el servicio del nivel vigente y responde su mensaje
5. Cada cambio de nivel queda registrado con su motivo

06 · CÓMO FUNCIONA · LA MÁQUINA DE ESTADOS

Tres estados, una regla y lo que responde cada uno



Nivel 1 · Full

valida, registra, analiza el riesgo y notifica

200	Nivel 1: OK
500	Nivel 1: Operación Full con Error

Nivel 2 · Degradado

valida y registra; difiere el resto

200	Nivel 2: OK
500	Nivel 2: Operación Limitada

Nivel 3 · Mínimo

no procesa: solo responde

200	Nivel 3: Operación al mínimo
503	Nivel 3: Sistema bajo mantenimiento, intente más tarde

07 · DEMO

Ahora, veámoslo funcionar en vivo

Vamos al panel de control: el mismo sistema desplegado en AWS, observado petición a petición.

▶ Ir al panel de control

1. La arquitectura en vivo
2. Errores a mano y la respuesta del Nivel 3
3. La racha que devuelve el sistema de a un nivel
4. La corrida oficial y su evidencia

08 · CÓMO FUNCIONA · CUANDO FALLA EL PROPIO MECANISMO

¿Y si lo que falla es la resiliencia?

Principio: si no conocemos la salud del sistema, operamos al mínimo. Nunca un error crudo.

SI…	EL SISTEMA…
DynamoDB no responde	responde Nivel 3 **«Sistema bajo mantenimiento»** (503) y lo registra
el servicio del nivel falla o tarda más de 3 s	responde el mensaje del Nivel 3, **cuenta el error** en el bloque y reinicia la racha
dos peticiones llegan a la vez a instancias distintas y las dos leen el mismo estado	deja que **solo una lo guarde** (escritura condicional por versión); la otra relee y recalcula. **Ninguna petición se pierde ni se numera dos veces**
CloudWatch no está disponible	**sigue respondiendo**; solo se pierde la visibilidad mientras dure

09 · CORRIDA OFICIAL · AWS · 02/11/2026

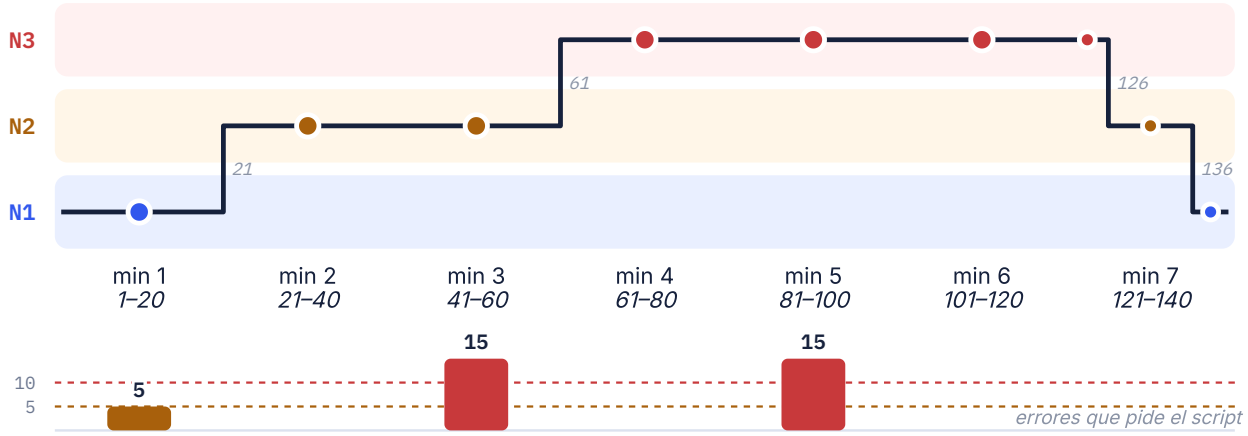
El script oficial en AWS: 1 → 2 → 3 → 2 → 1

✓ 140 de 140 peticiones

✓ Pasa por los 3 niveles

✓ Sin saltos

✓ Termina en Nivel 1



CLIENTE = SERVIDOR

140/140

peticiones con el mismo nivel y status

LATENCIA MEDIA (K6)

167 ms

p95 de 182 ms · router 53 ms

CAMBIOS REGISTRADOS

4

en las peticiones 21 · 61 · 126 · 136

«MANTENIMIENTO»

15

los errores del minuto 5, en N3

CIERRE

Responder siempre, aunque sea con menos

PRIORIZAMOS

Disponibilidad: el sistema nunca deja de responder

DECIDIMOS

Degradar de golpe y recuperar con calma, de a un nivel

DEMOSTRAMOS

1 → 2 → 3 → 2 → 1 en AWS, 140 de 140 peticiones

Gracias · ¿Preguntas?

Group 2 · Daniel Mazo · Freddy Aparicio · Camilo Céspedes